



# Universidad Nacional Autónoma de México

Facultad de Ingeniería

# Bitácora S11

Persistencia políglota y caché en Mexi Banco


**Asignatura:** Sistemas Distribuidos
**Profesor:** Oscar Manuel Ruiz Hurtado
**Grupo:** 02
**Semestre:** 2027-1
**Integrante:** Arroyo Onofre Leonardo

# Objetivo

En esta sesión se trabajó con persistencia políglota en Mexi Banco: conservar el dinero en PostgreSQL, usar Redis para mostrar una copia del saldo y guardar los avisos en MongoDB. La idea fue darle a cada dato un almacenamiento adecuado, y comprobar con mediciones qué cambia al agregar la caché. Los resultados muestran menos lecturas a Postgres, mientras que los avisos pueden guardarse con campos distintos según su canal sin cambiar las reglas del saldo.

# 1 Preparación y punto de partida

Primero se levantó el proyecto con Docker Compose. Para esta sesión ya utilicé Git Bash en vez de PowerShell, con lo que pude ejecutar los ciclos y comandos de la guía en la misma terminal. Se conservaron las tres instancias de cuenta para comprobar después que la caché fuera compartida.

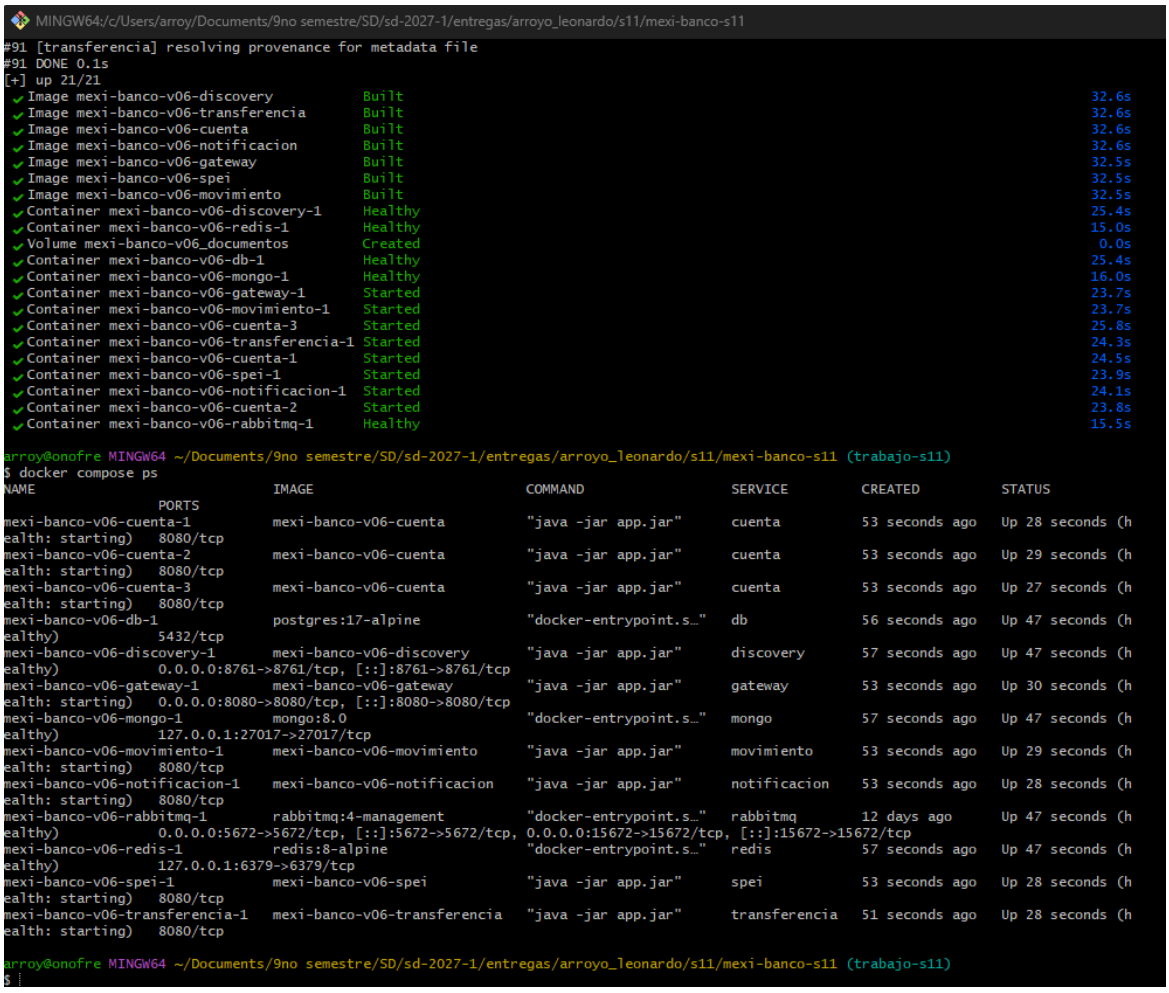



Figura 1: Construcción del proyecto y consulta de los contenedores con Docker Compose.

Después se comprobó la conexión de los dos almacenes nuevos. Redis respondió PONG y el conteo inicial de avisos de MongoDB fue 0. Esto permitió comenzar con una referencia clara antes de guardar documentos.

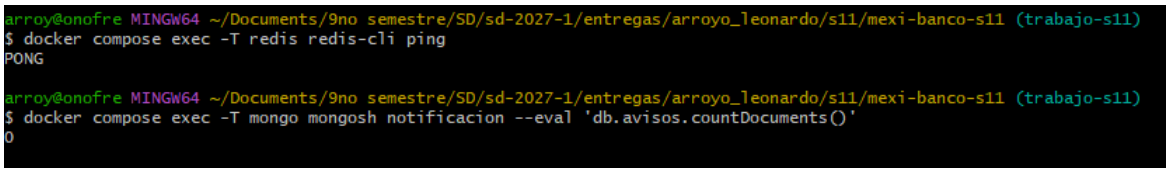



Figura 2: Respuesta PONG de Redis y colección avisos inicialmente vacía.

Se crearon las cuentas de Ana, Beto y Carla, con terminaciones 001, 002 y 003 y saldos iniciales de 1000, 500 y 300. Luego se transfirieron 200 de Ana a Beto. La respuesta fue 201 Created y la transferencia 35 quedó COMPLETADA. Así, Ana comenzó las consultas de referencia con 800 y Beto con 700.




Figura 3: Creación de las tres cuentas con sus saldos iniciales.

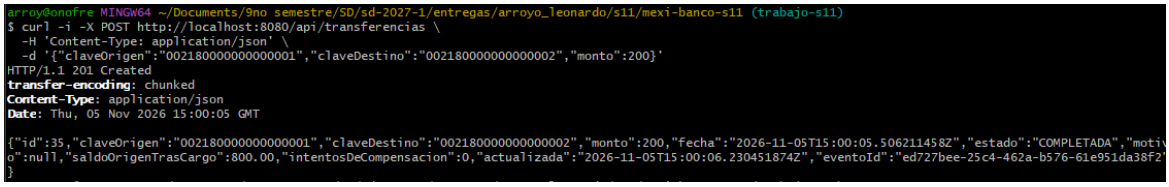



Figura 4: Transferencia inicial de 200 con estado COMPLETADA.

Antes de habilitar el uso de la caché se consultó diez veces el saldo de Ana. Las diez respuestas mostraron 800, pero cada una tuvo un leidaEn diferente. La diferencia del contador de logs fue de diez lecturas a Postgres. Redis seguía con cero hits y cero misses: no se estaba consultando la copia, por lo que tampoco había aciertos ni fallos de caché.

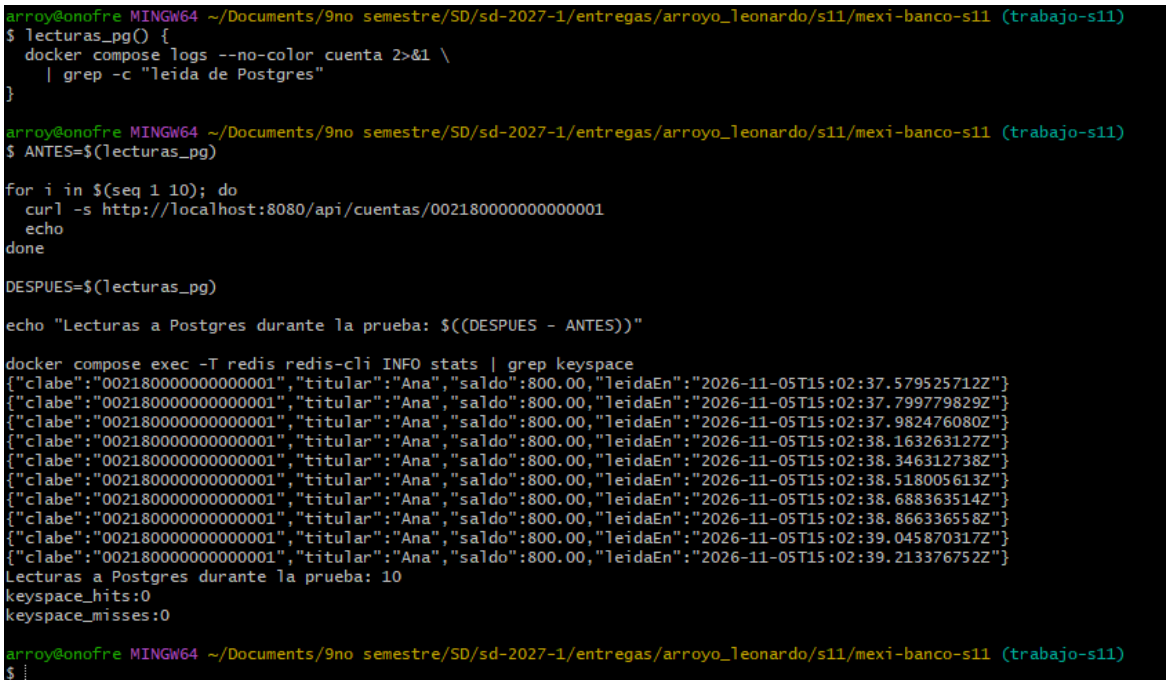



Figura 5: Diez consultas que producen diez lecturas a Postgres antes de usar la caché.

# 2 Caché compartida del saldo

La caché se aplicó solamente a la consulta que muestra el saldo. En ConsultaDeSaldo, @Cacheable usa el nombre cuentas y la CLABE como llave. Si Redis tiene el valor, Spring devuelve la copia; si no lo tiene, ejecuta el método, consulta Postgres y guarda el resultado. Por eso el log leida de Postgres permite contar las ejecuciones reales del método y no todas las peticiones HTTP.

```
@Cacheable(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
public CuentaVista consultar(String clabe) {
        // Esta linea sale en el log SOLO cuando se lee Postgres. Con la cache puesta, en un hit no aparece
    .
        log.info("[consulta] {} leida de Postgres", clabe);
        return cuentas.findByClabe(clabe)
                .map(CuentaVista::de)
                .orElseThrow(() -> new NoEncontradoException("No existe la CLABE " + clabe));
}
```

ConfiguracionCache habilita las anotaciones con @EnableCaching. La configuración da a cada entrada un TTL de 30 segundos, evita guardar valores nulos y serializa CuentaVista como JSON con el ObjectMapper de Spring. El valor incluye clabe, titular, saldo y leidaEn. La llave resultante en Redis tiene la forma cuentas::<clabe>.

```
RedisCacheConfiguration cuentas =
        RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(ttl)
                .disableCachingNullValues()
                .serializeValuesWith(
                        SerializationPair.fromSerializer(
                                new Jackson2JsonRedisSerializer<>(
                                        json,
                                        CuentaVista.class
                                )
                        )
                );
```

El administrador de caché se configura con transactionAware(). Esto difiere las invalidaciones transaccionales hasta después de confirmar los cambios en Postgres. En CuentaService se agregó @CacheEvict tanto a cargar como a abonar para borrar la entrada de la cuenta modificada. La lectura que autoriza el cargo permanece en CuentaService.consultar y va al repositorio de Postgres, no a ConsultaDeSaldo.

```
@CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
@Transactional
public SaldoAplicado cargar(String clabe, BigDecimal monto, String clave)

@CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
@Transactional
public SaldoAplicado abonar(String clabe, BigDecimal monto, String clave)
```

Las firmas anteriores señalan dónde se colocó la invalidación. La separación entre mostrar y autorizar es importante: se acepta que una pantalla vea una copia por unos segundos, pero no que esa copia decida si alcanza el dinero para un cargo.

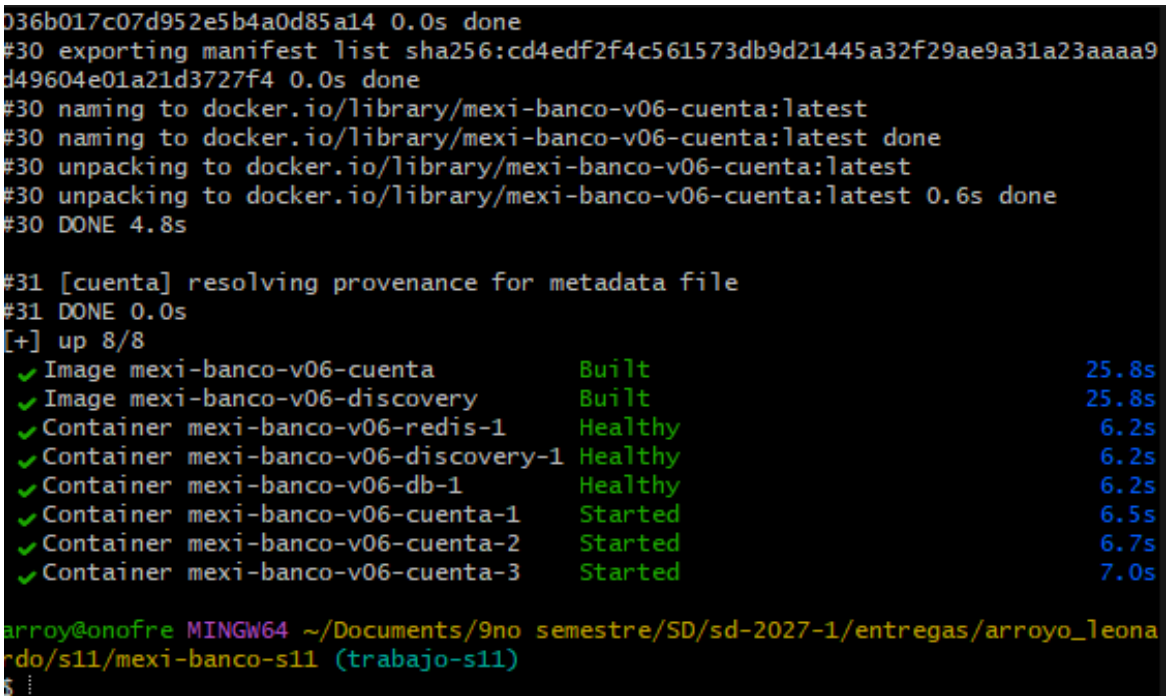



Figura 6: Reconstrucción de cuenta después de configurar la caché.

Para probar que las instancias compartían Redis se borró la llave de Ana y se hicieron seis consultas. Las cabeceras X-Instancia cambiaron entre tres identificadores, pero las respuestas conservaron el mismo leidaEn y sólo se registró una lectura a Postgres. Redis devolvió un JSON legible y un TTL de 23 segundos. La primera consulta llenó la copia y las siguientes pudieron reutilizarla aunque llegaran a otra instancia.

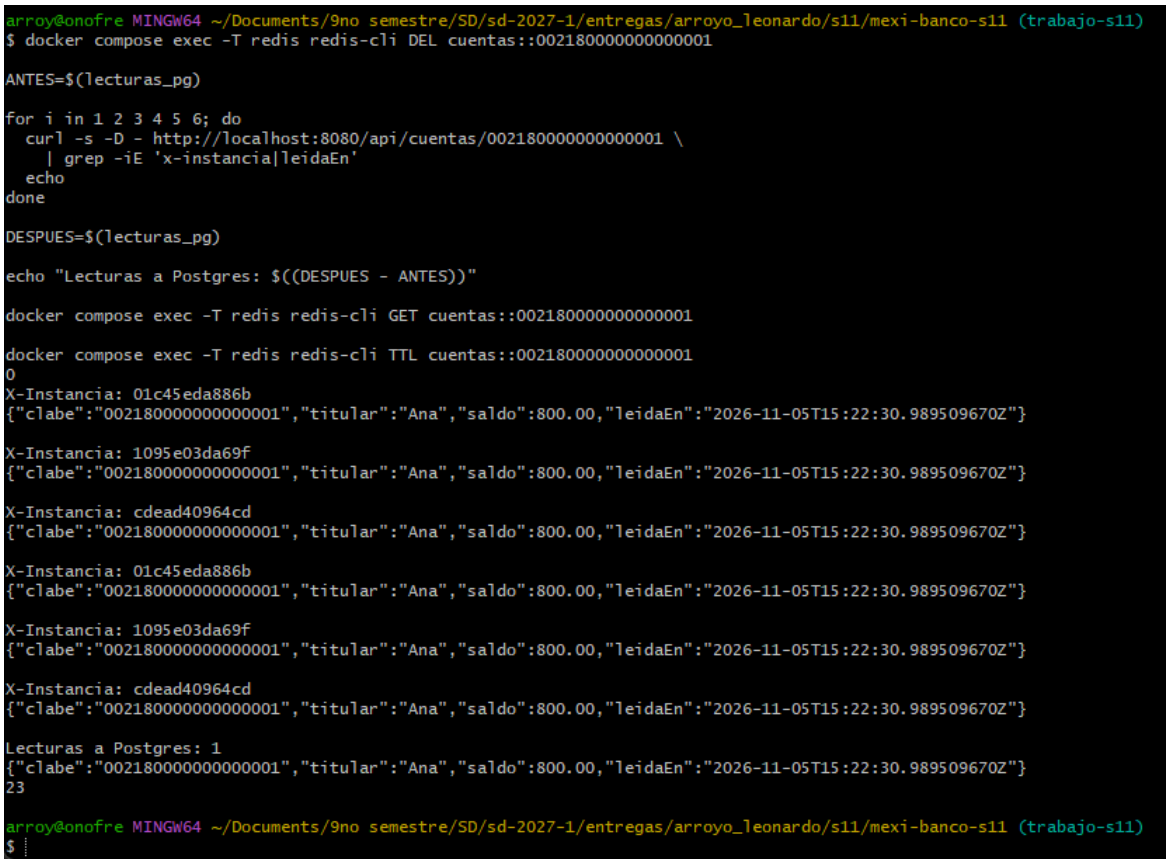



Figura 7: Seis respuestas de distintas instancias con una sola lectura a Postgres, JSON y TTL.

Luego se transfirieron 10 de Ana a Beto. La operación respondió 201 y Ana pasó de 800 a 790. En MONITOR se observaron GET, SET con PX 30000 y DEL de las dos cuentas. El DEL elimina la copia anterior; la siguiente consulta puede volver a leer el saldo confirmado y guardar una entrada nueva.




Figura 8: Transferencia de 10 completada para probar la invalidación.

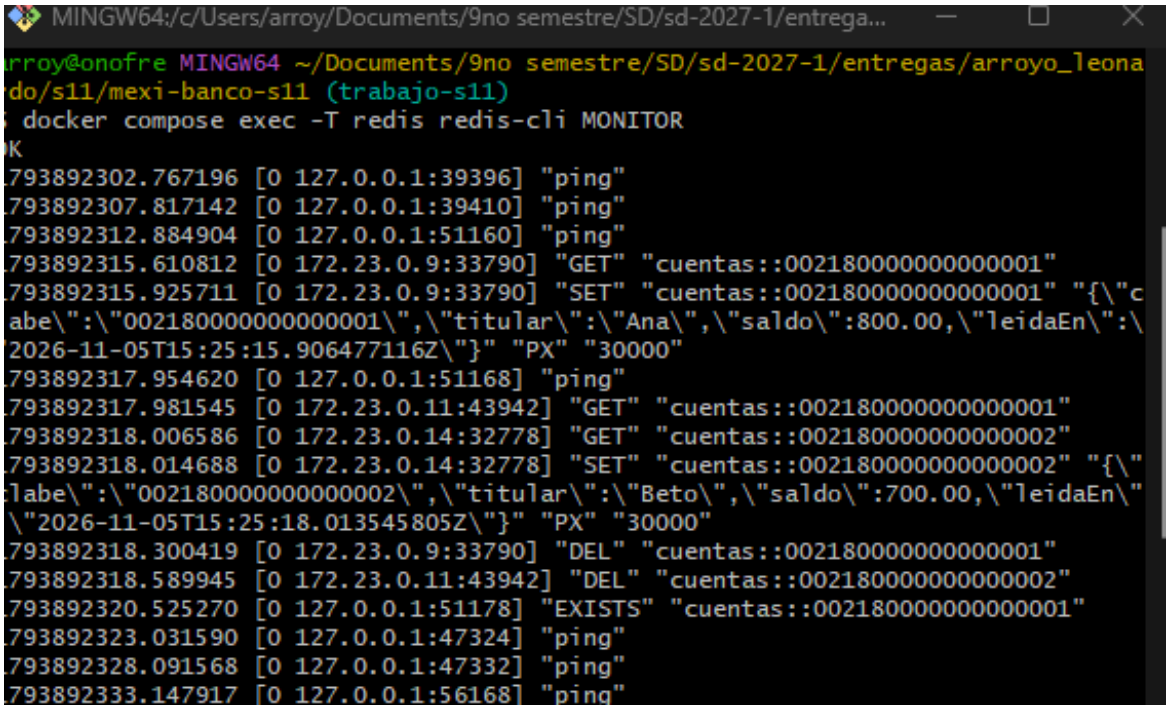



Figura 9: MONITOR muestra la lectura, el llenado y el borrado de las llaves de origen y destino.

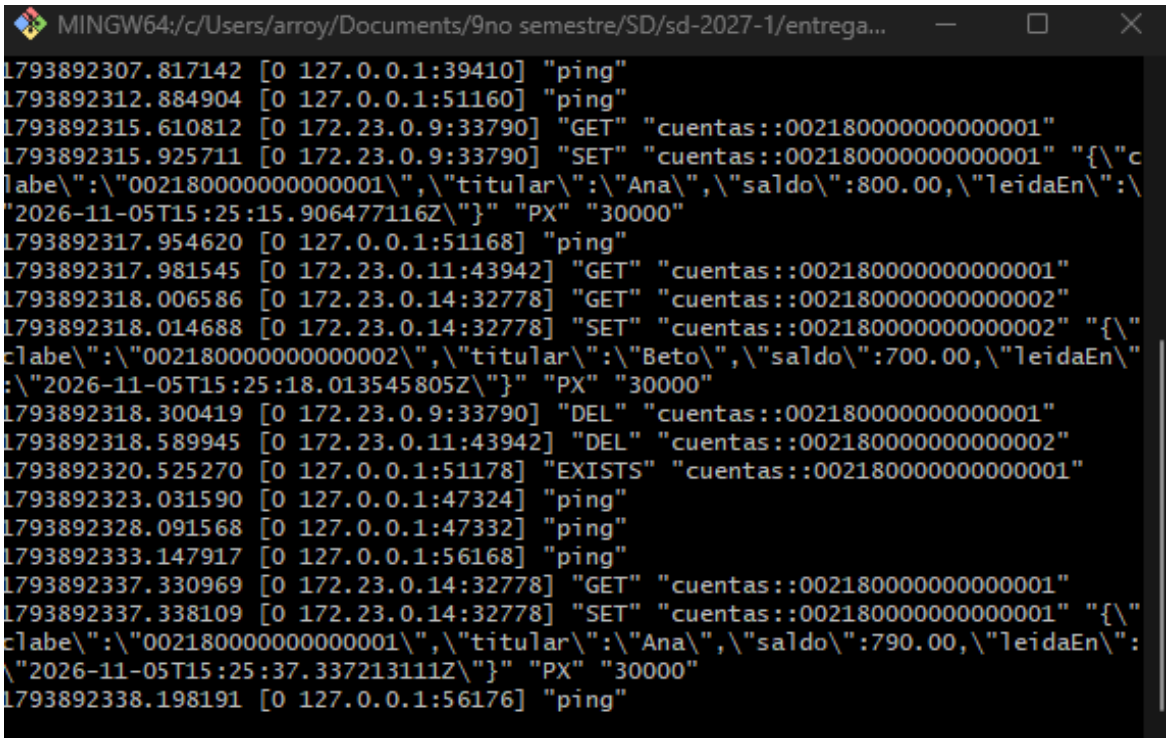



Figura 10: Nueva lectura del saldo 790 y llenado de la caché después de invalidarla.

# 3 Medición del hit ratio

El hit ratio es la proporción de consultas a Redis que encontraron la llave: hits dividido entre hits más misses. Antes de medir se reiniciaron los contadores. En el escenario A se hicieron cien consultas seguidas del saldo de Ana. El resultado fue 99 hits, 1 miss y una lectura a Postgres: 99 dividido entre 100 da 99 %.

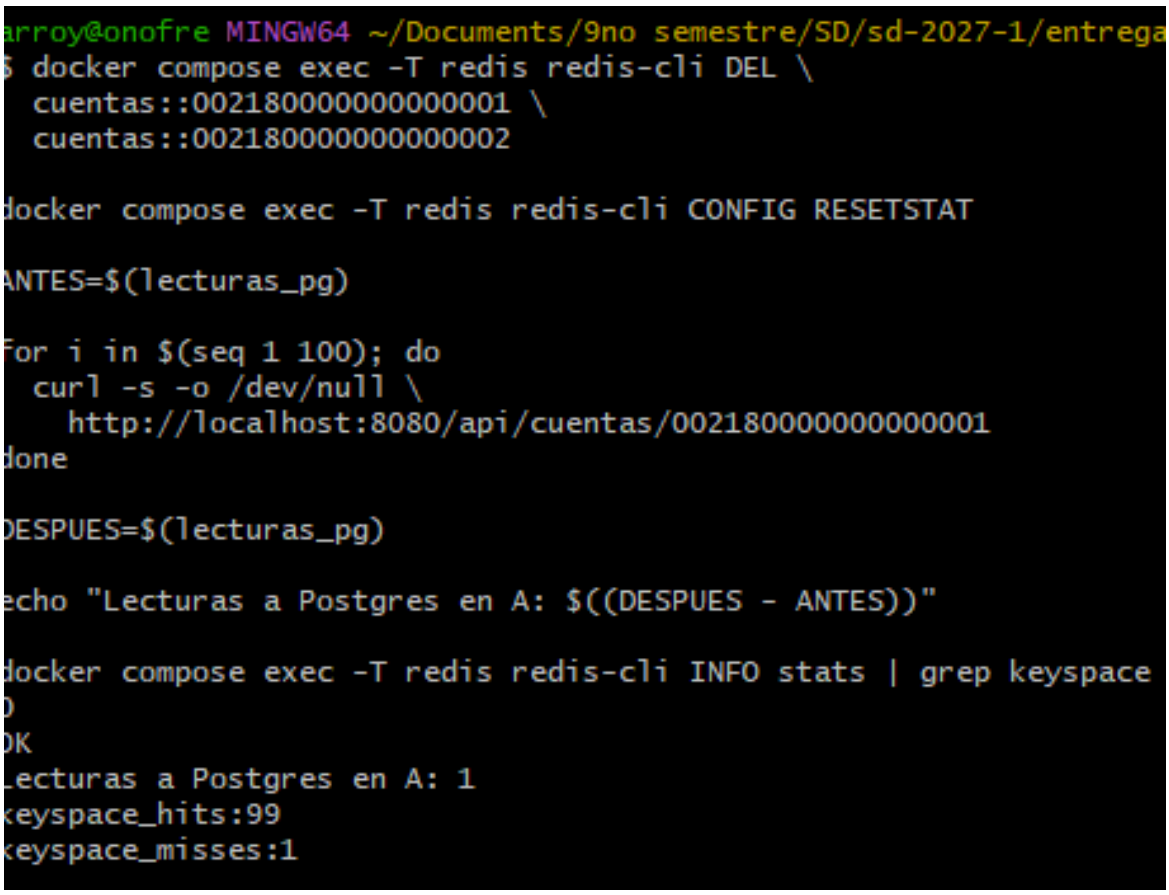



Figura 11: Escenario A con 99 hits, 1 miss y una lectura a Postgres.

En el escenario B se intercaló una transferencia de un peso cada diez consultas. Se obtuvieron 100 hits, 20 misses y 20 lecturas a Postgres. El hit ratio fue 100 dividido entre 120, equivalente a 83.33 %. No se divide entre cien porque el contador de Redis incluye también las consultas que el flujo de transferencia hizo a las cuentas; no cuenta exclusivamente los GET del ciclo.

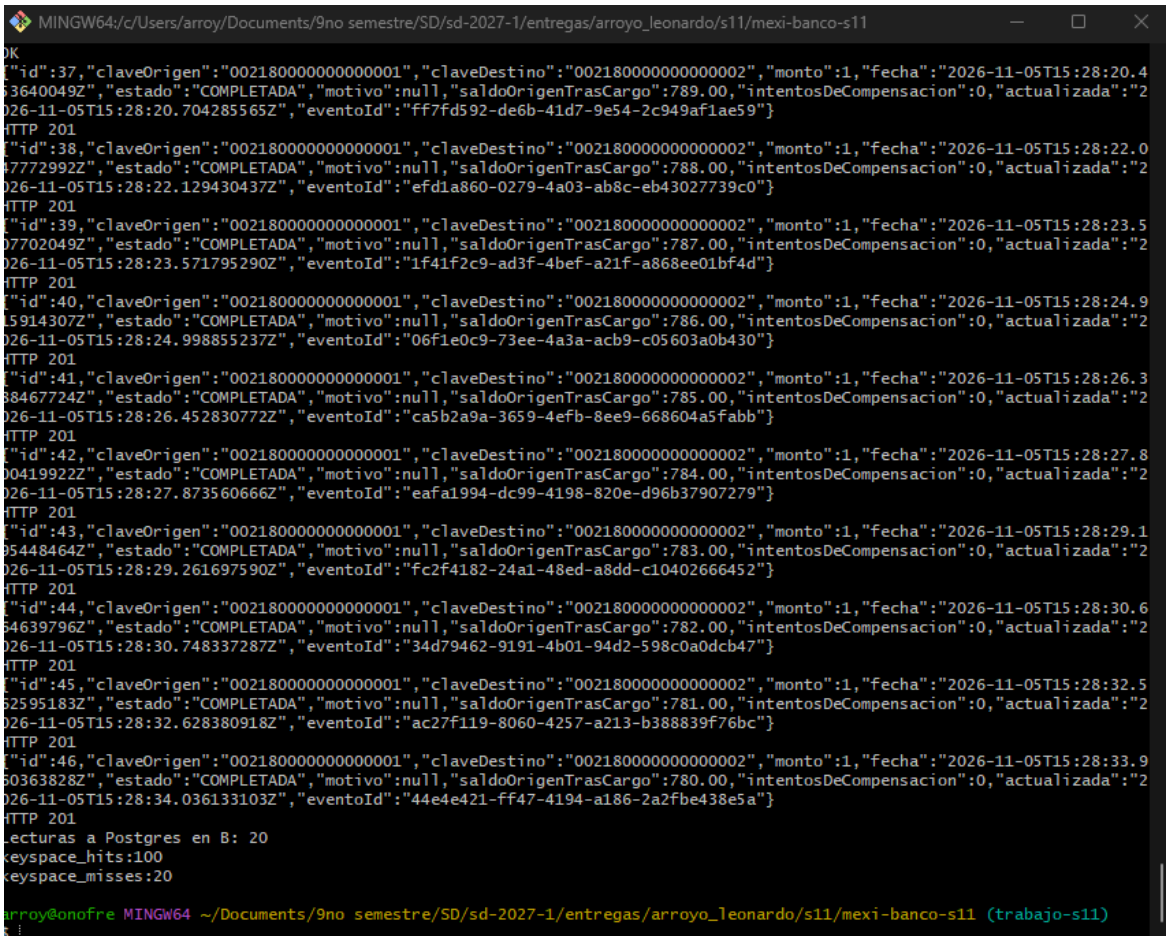



Figura 12: Escenario B con transferencias intercaladas, 100 hits y 20 misses.

| Escenario | Hits | Misses | Hit ratio | Lecturas PG |
|---|---|---|---|---|
| A | 99 | 1 | 99 % | 1 |
| B | 100 | 20 | 83.33 % | 20 |

¿Por qué bajó? Cada transferencia modifica el origen y el destino y borra sus copias. Las consultas posteriores necesitan recuperarlas desde Postgres. La caché funciona mejor cuando se repite la lectura sin tantos cambios; al aumentar las escrituras, también aumenta la invalidación. Esto es parte del costo de mantener una copia útil y no quedarse mostrando siempre el saldo anterior.

# 4 Latencia y lecturas a Postgres

Ahora se midieron cien consultas por corrida, tres veces con caché y tres sin ella. Se registraron promedio, mediana y p95 en milisegundos, junto con las lecturas a Postgres. El p95 reportado corresponde al tiempo que alcanzó el 95 % de las consultas de cada corrida, según el ordenamiento usado en la medición.

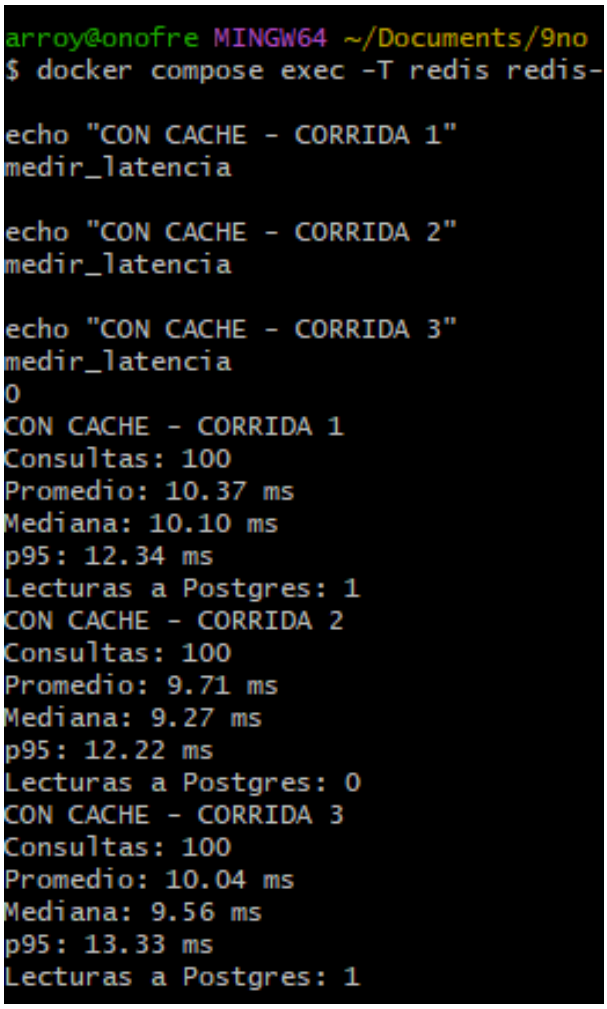



Figura 13: Tres corridas completas con caché y sus lecturas a Postgres.

Para comparar se reinició cuenta con CACHE_TIPO=none. Esto deshabilita el uso de la caché en la aplicación; no equivale a apagar el contenedor de Redis. Se repitió el mismo grupo de cien consultas y se contaron las lecturas del intervalo de cada prueba.

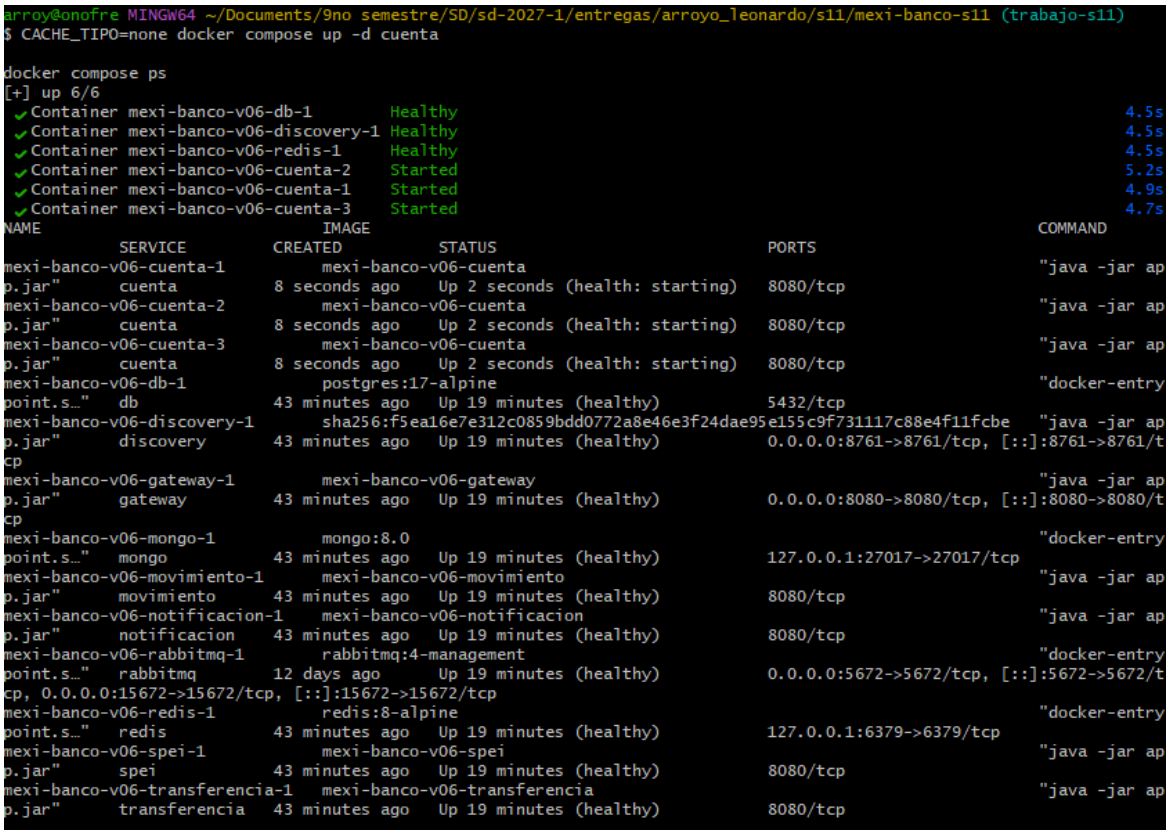



Figura 14: Cambio a CACHE_TIPO=none para realizar la comparación.

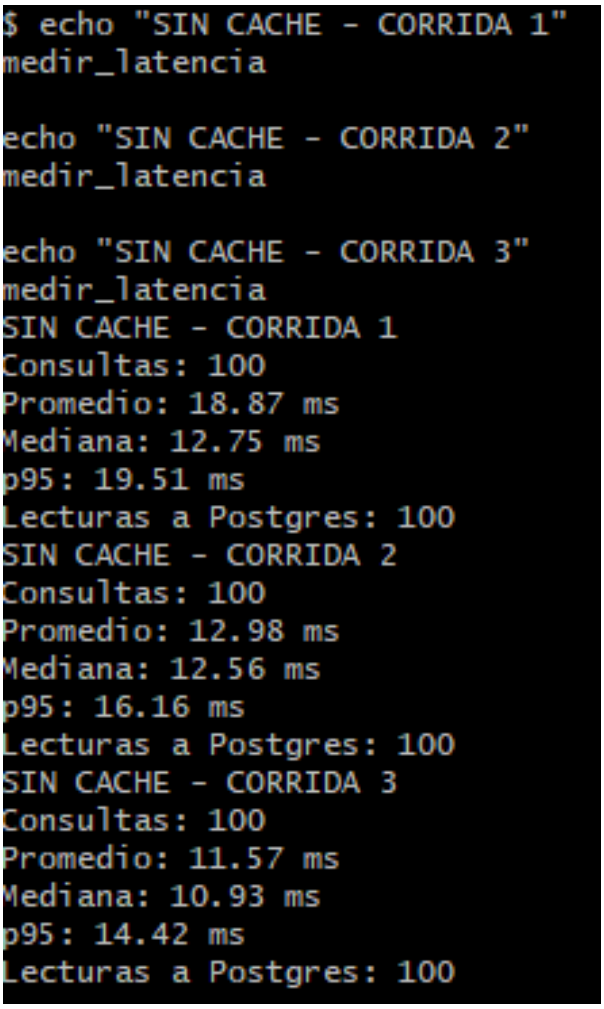



Figura 15: Tres corridas sin caché con cien lecturas a Postgres cada una.

| Modo | Corrida | Media ms | Mediana ms | p95 ms | Lecturas PG |
|---|---|---|---|---|---|
| Con caché | 1 | 10.37 | 10.10 | 12.34 | 1 |
| Con caché | 2 | 9.71 | 9.27 | 12.22 | 0 |
| Con caché | 3 | 10.04 | 9.56 | 13.33 | 1 |
| Sin caché | 1 | 18.87 | 12.75 | 19.51 | 100 |
| Sin caché | 2 | 12.98 | 12.56 | 16.16 | 100 |
| Sin caché | 3 | 11.57 | 10.93 | 14.42 | 100 |

Con caché los promedios quedaron entre 9.71 y 10.37 ms; sin caché, entre 11.57 y 18.87 ms. Hay una mejora de latencia en estas corridas, aunque la primera sin caché tuvo más variación y no conviene tomarla como una diferencia constante. La evidencia más clara fue la carga de lectura: en las 300 consultas con caché se registraron dos lecturas a Postgres, frente a 300 sin caché. Son 298 menos, aproximadamente una reducción de 99.33 % en estas consultas instrumentadas.

La corrida con cero lecturas significa que la copia ya estaba disponible durante ese intervalo. No es que Postgres nunca se consulte: el llenado ocurrió antes o en otra corrida, y la entrada también puede vencer. El beneficio observado fue tanto responder más rápido

como evitar trabajo repetido en la base; lo segundo deja margen para atender más consultas, aunque estas pruebas secuenciales no miden por sí solas la capacidad máxima del sistema.

# 5 Avisos como documentos de MongoDB

Después se cambió la persistencia de avisos. En BitacoraDeAvisos, cada aviso se construye como un Document. Los campos comunes son eventoId, transferenciaId, clabe, tipo, monto, saldo y fecha. El monto y el saldo usan Decimal128 en vez de double para conservar la representación decimal del dinero; la fecha se toma del evento mediante Date.from(evento.fecha()).

```
Document aviso = new Document(
                "_id",
                evento.eventoId() + ":" + destinatario.clabe()
)
                .append("eventoId", evento.eventoId().toString())
                .append("transferenciaId", evento.transferenciaId())
                .append("clabe", destinatario.clabe())
                .append("tipo", destinatario.tipo())
                .append("monto", new Decimal128(evento.monto()))
                .append("saldo", new Decimal128(destinatario.saldo()))
                .append("fecha", Date.from(evento.fecha()));
```

El documento agrega los campos que corresponden al canal. SMS tiene telefono y texto; CORREO tiene correo, asunto y cuerpo; PUSH tiene token, titulo y cuerpo. El switch usa los tipos de Contacto para construir cada forma. Así no es necesario llenar con valores nulos todos los campos de los otros canales.

```
switch (contacto) {
        case Contacto.Sms sms -> aviso
                        .append("canal", "SMS")
                        .append("telefono", sms.telefono())
                        .append("texto", texto);

        case Contacto.Correo correo -> aviso
                        .append("canal", "CORREO")
                        .append("correo", correo.correo())
                        .append("asunto", asunto(destinatario.tipo()))
                        .append("cuerpo", texto);

        case Contacto.Push push -> aviso
                        .append("canal", "PUSH")
                        .append("token", push.token())
                        .append("titulo", asunto(destinatario.tipo()))
                        .append("cuerpo", texto);
}
```

En el consumidor se reemplazó el guardado anterior por nuevos = bitacora.registrar(evento). Primero se guardan los avisos y luego se envía basicAck al broker. Si el guardado falla, el mensaje se reencola. No se escribe una copia adicional de estos avisos en Postgres, de modo que MongoDB queda como la bitácora consultada por este flujo.

Se reconstruyó notificacion y apareció el log indice clabe_fecha listo en avisos. Después se hicieron dos transferencias de 50: una de Ana a Beto y otra de Ana a Carla. Ambas respondieron 201 y generaron cuatro avisos: dos de salida de Ana, uno de entrada de Beto y uno de entrada de Carla.

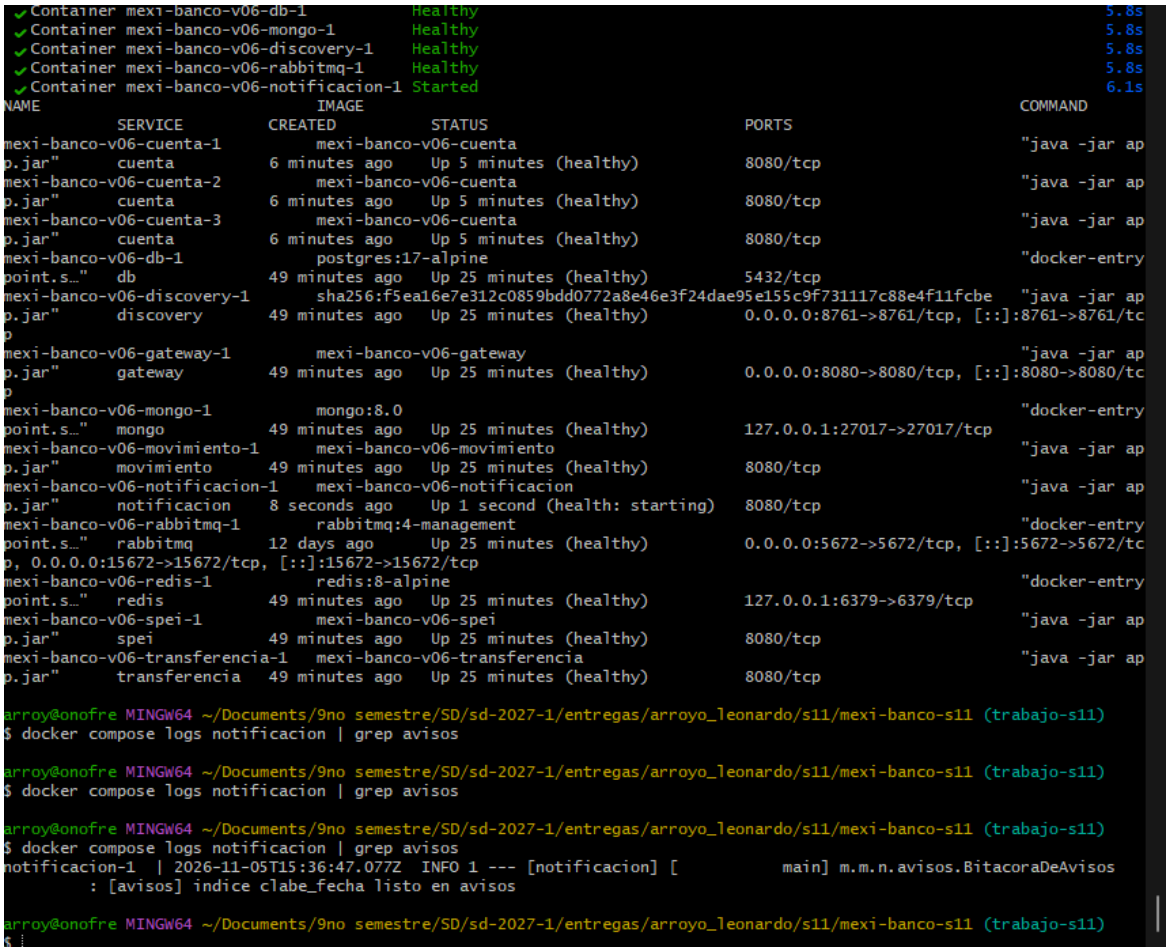



Figura 16: Reconstrucción de notificacion y log de creación del índice clabe_fecha.

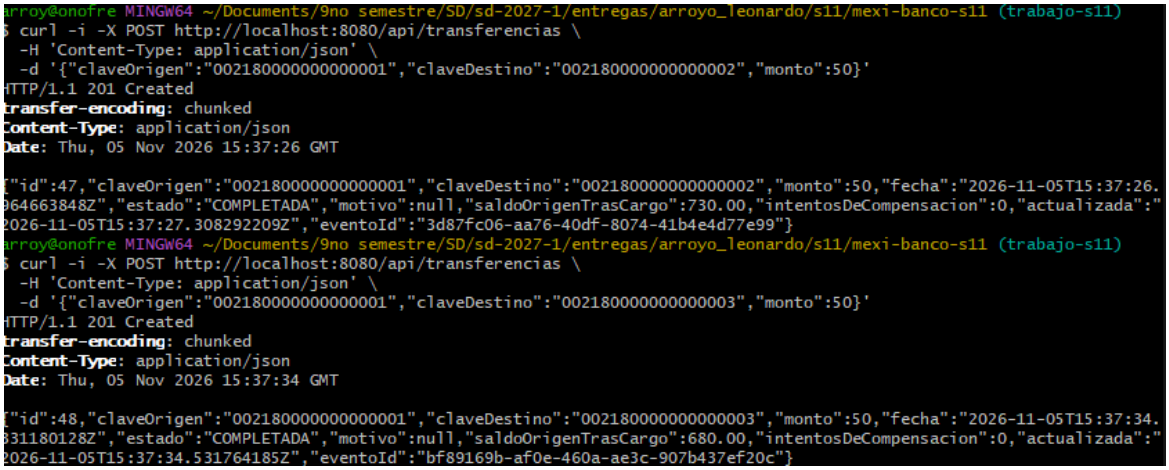



Figura 17: Transferencias de 50 hacia Beto y Carla con respuesta 201.

Al consultar la colección se vieron las tres formas. Ana recibió SMS, Beto CORREO y Carla PUSH. En los documentos se observan Decimal128 para monto y saldo y un _id con la forma eventoId:clabe. El aviso conserva el saldo resultante de ese evento; no es una consulta del saldo actual de la cuenta.

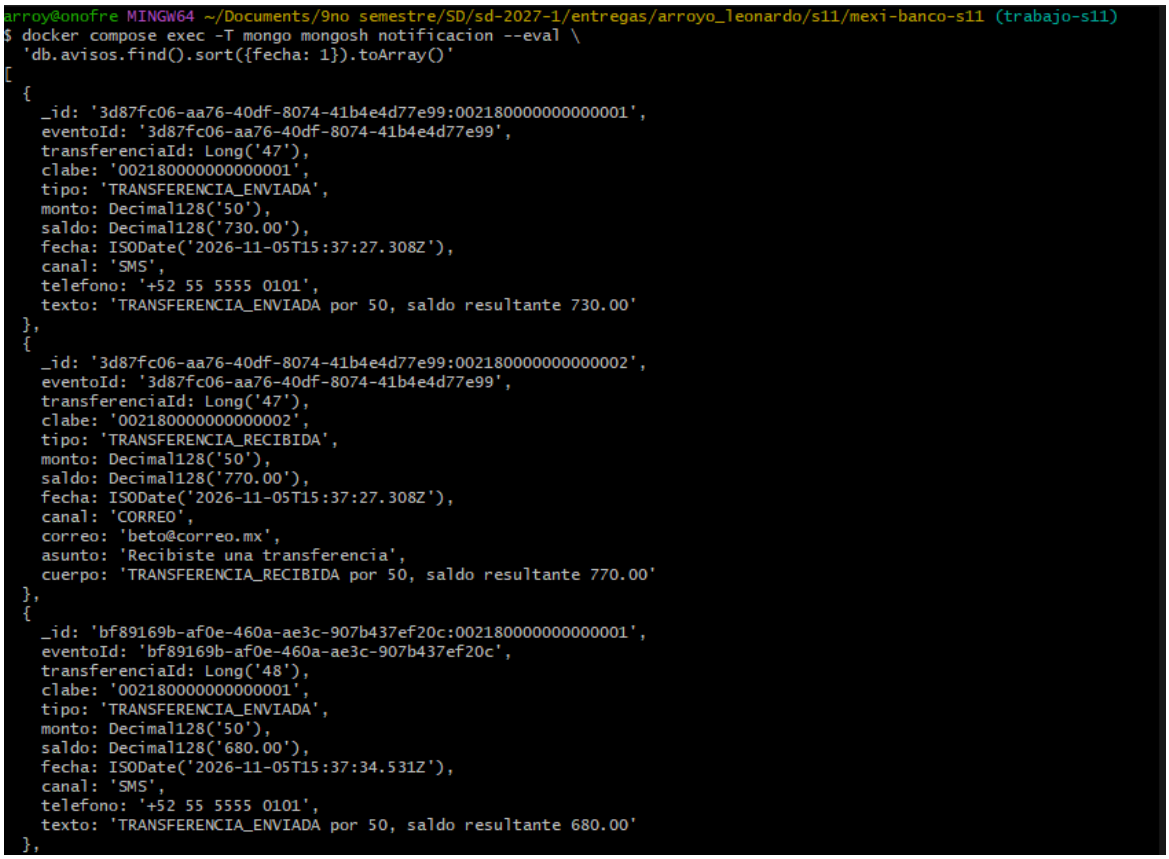



Figura 18: Documentos SMS y CORREO con campos comunes y dinero en Decimal128.

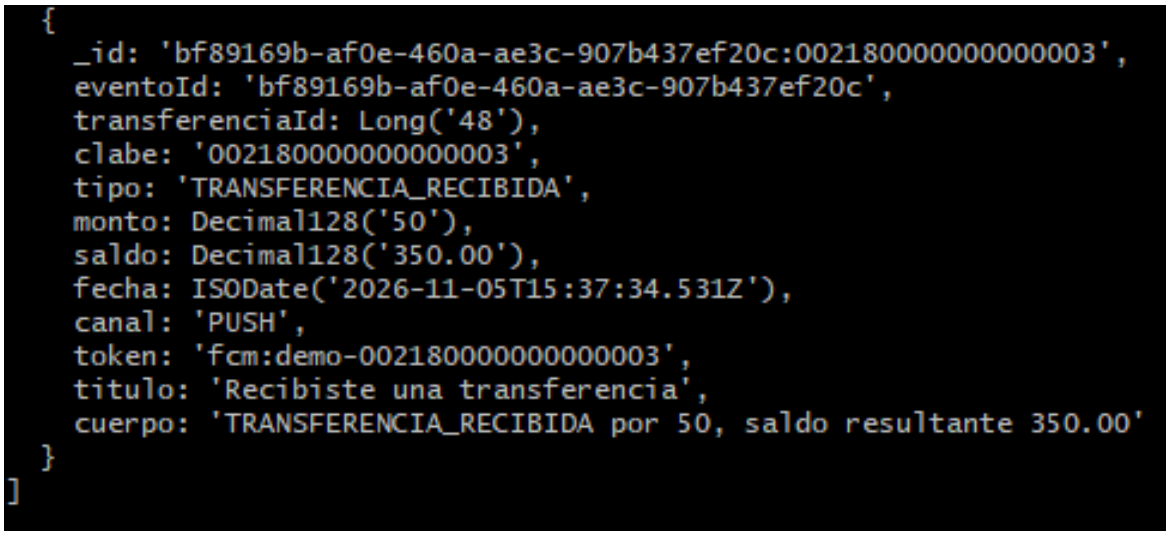



Figura 19: Documento PUSH de Carla con token, titulo y cuerpo.

También se filtraron los documentos con telefono existente. El resultado mostró los dos SMS de Ana y sus campos de teléfono. Esto permite consultar una característica propia de un canal sin exigir que todos los documentos tengan esa misma forma.




Figura 20: Filtro por existencia de telefono que selecciona los avisos SMS.

# 6 Consulta de los diez avisos más recientes

Se generaron doce transferencias de un peso de Ana a Beto para tener más avisos que el límite de consulta. El método ultimos filtra por CLABE, ordena fecha de manera descendente y aplica limit(ULTIMOS), cuyo valor es diez. Este es el método que sirve la consulta de la bitácora por cuenta.

```
public List<Document> ultimos(String clabe) {
    Query consulta = Query.query(
                                            Criteria.where("clabe").is(clabe)
                                )
                                .with(Sort.by(Sort.Direction.DESC, "fecha"))
                                .limit(ULTIMOS);

                return mongo.find(consulta, Document.class, COLECCION);
        }
```

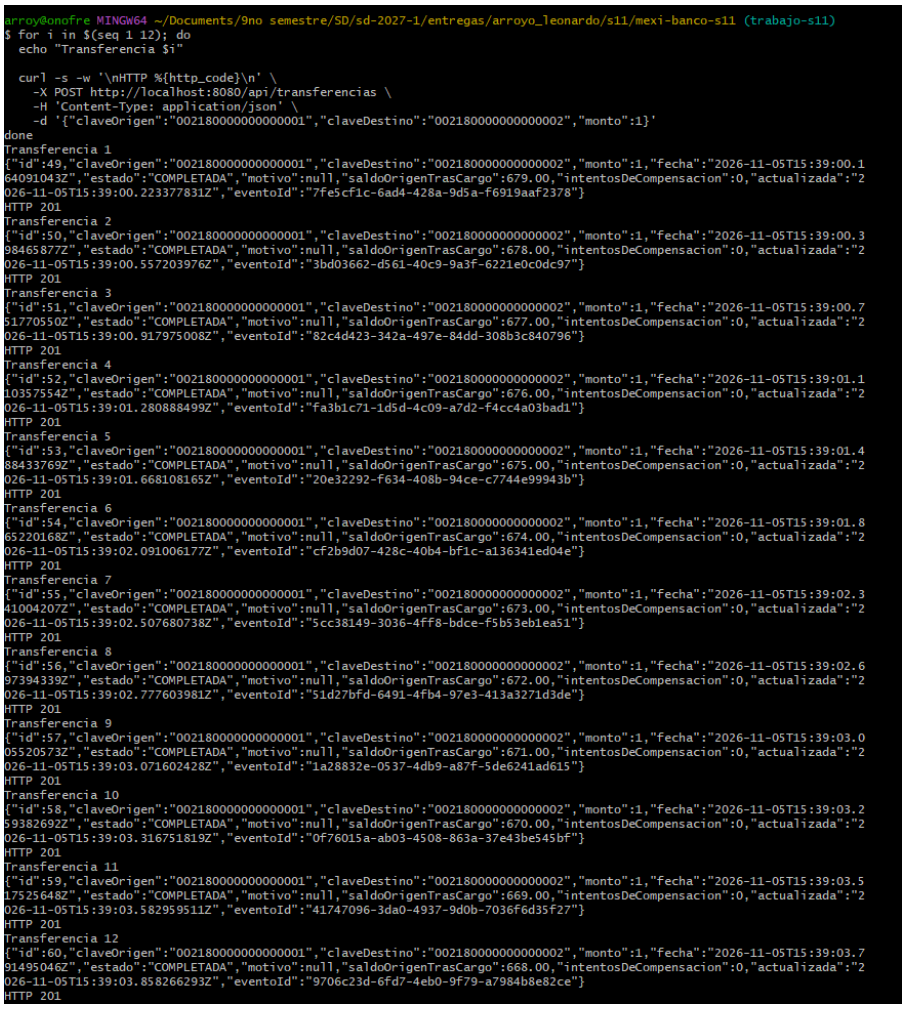



Figura 21: Doce transferencias pequeñas para superar el límite de diez avisos.

La consulta en MongoDB devolvió Cantidad: 10 y documentos del canal CORREO para la CLABE de Beto. La salida va del más reciente al más viejo y muestra transferencias de la 60 a la 51. No se borraron los anteriores: el límite restringe lo que se devuelve, no lo que se conserva en la colección.

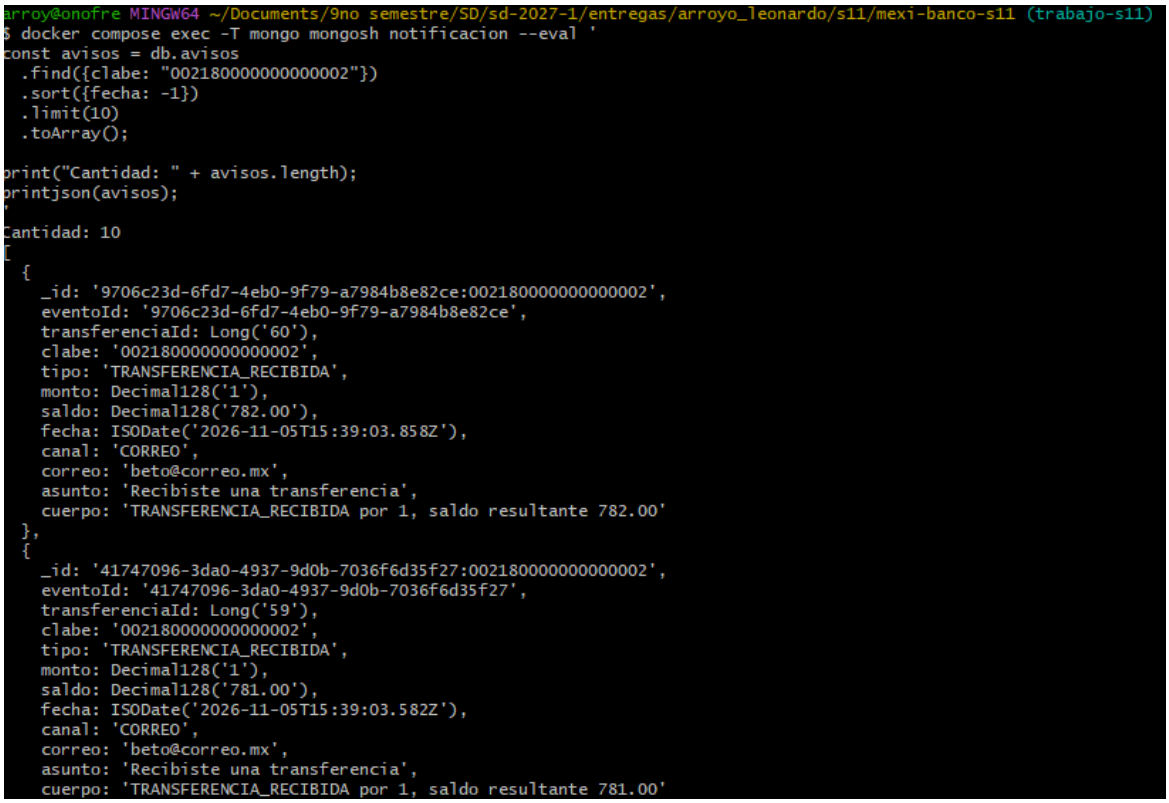



Figura 22: Consulta con límite de diez y primeros documentos en orden descendente.

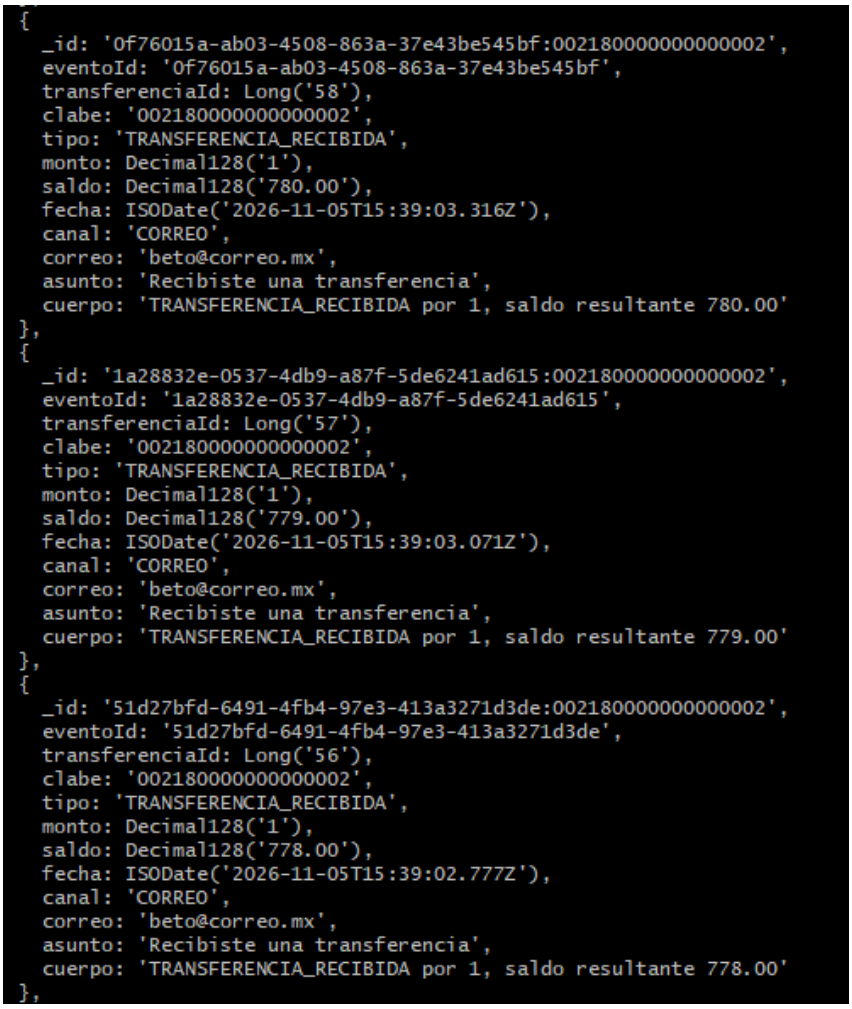



Figura 23: Continuación de la consulta con avisos de Beto del canal CORREO.

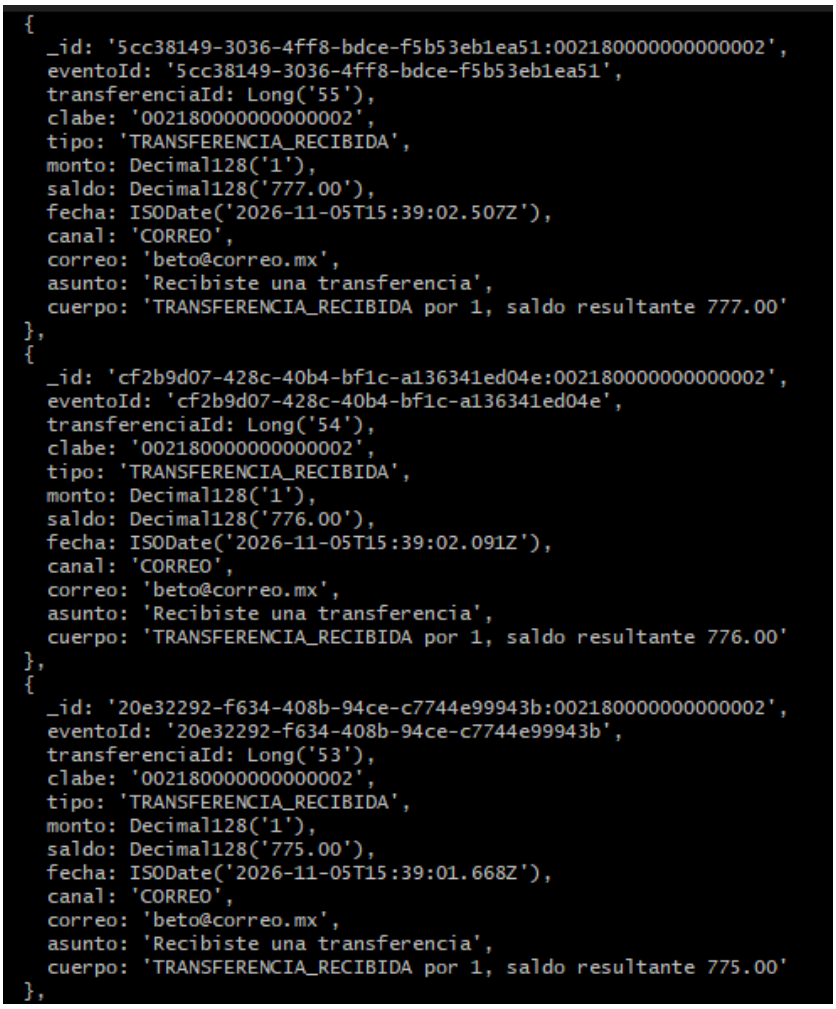



Figura 24: Continuación de los avisos ordenados por fecha descendente.

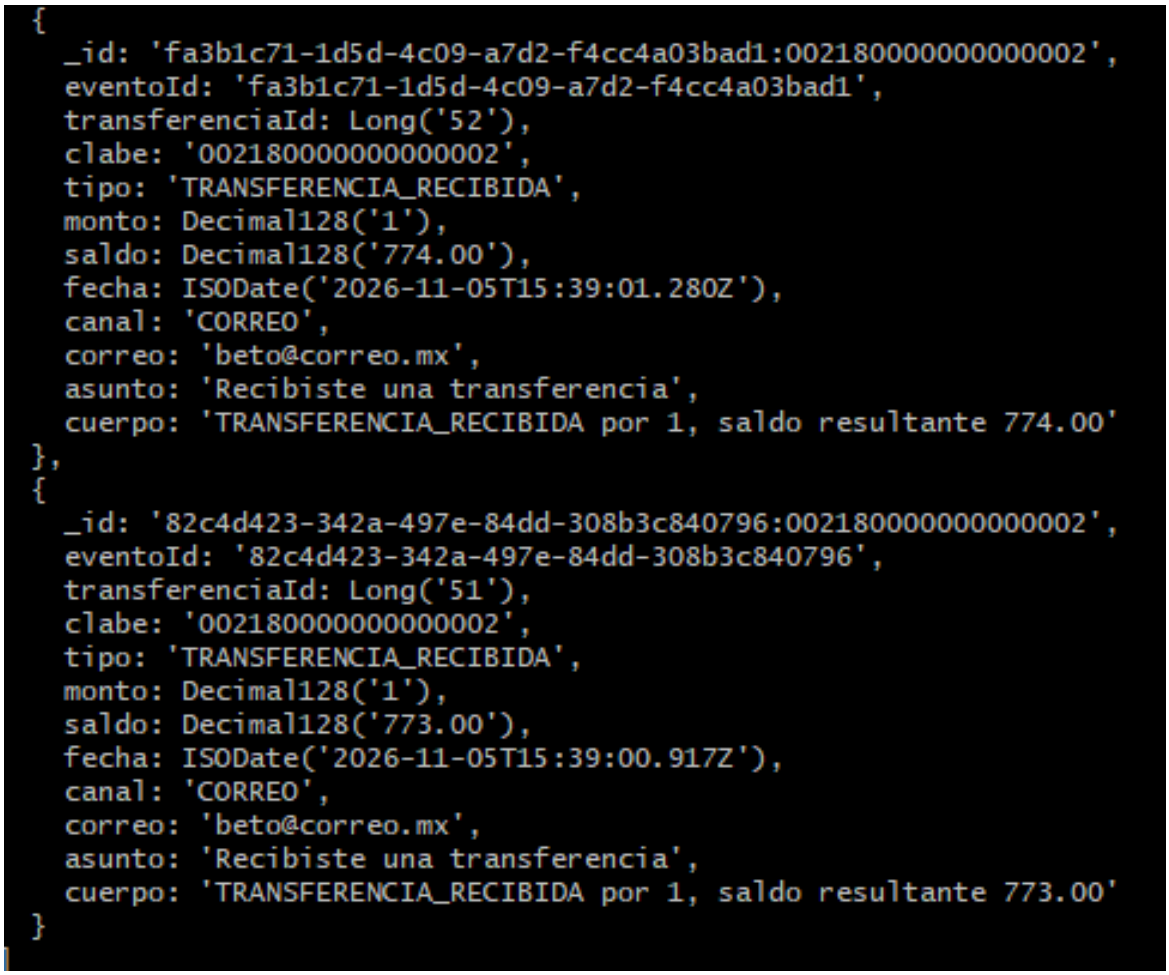



Figura 25: Últimos dos documentos del resultado limitado a diez.

# 7 Índice y planes de ejecución

El índice compuesto se llamó clabe_fecha, con clabe ascendente y fecha descendente. Su orden coincide con la consulta: primero se localizan los avisos de una cuenta y dentro de ellos se recorren los más recientes. Se crea al arrancar notificacion mediante ensureIndex; la consulta de índices mostró tanto _id_ como clabe_fecha.

```
@EventListener(ApplicationReadyEvent.class)
public void crearIndice() {
        String nombre = mongo.indexOps(COLECCION).ensureIndex(
                        new Index()
                                        .on("clabe", Sort.Direction.ASC)
                                        .on("fecha", Sort.Direction.DESC)
                                        .named("clabe_fecha")
        );

        log.info("[avisos] indice {} listo en {}", nombre, COLECCION);
}
```

```
arroy@onofre MINGW64 ~/Documents/9no semestre/SD/sd-2027-1/entregas/arroyo_leonardo/s11/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval \
  'db.avisos.getIndexes()'
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { clabe: 1, fecha: -1 }, name: 'clabe_fecha' }
]
```

Figura 26: Índices de la colección avisos con clabe_fecha y su orden compuesto.

Se comparó la misma consulta usando explain con executionStats. Con el índice aparecieron las etapas LIMIT, FETCH e IXSCAN: se examinaron diez llaves y diez documentos para devolver diez avisos. Después se forzó el recorrido natural con hint({$natural: 1}). En ese caso aparecieron COLLSCAN y SORT: se examinaron los 28 documentos de la colección, se encontraron 13 de Beto y se devolvieron los diez más recientes.

```
executionStats: {
  executionSuccess: true,
  nReturned: 10,
  executionTimeMillis: 12,
  totalKeysExamined: 10,
  totalDocsExamined: 10,
  executionStages: {
    isCached: false,
    stage: 'LIMIT',
    nReturned: 10,
    executionTimeMillisEstimate: 0,
    works: 11,
    advanced: 10,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    limitAmount: 10,
    inputStage: {
      stage: 'FETCH',
      nReturned: 10,
      executionTimeMillisEstimate: 0,
      works: 10,
      advanced: 10,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 0,
      docsExamined: 10,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 10,
        executionTimeMillisEstimate: 0,
        works: 10,
        advanced: 10,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 0,
        keyPattern: { clabe: 1, fecha: -1 },
        indexName: 'clabe_fecha',
        isMultiKey: false,
        multiKeyPaths: { clabe: [], fecha: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          clabe: [ '["002180000000000002", "002180000000000002"]' ],
          fecha: [ '[MaxKey, MinKey]' ]
        },
        keysExamined: 10,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0
      }
    }
  }
```

Figura 27: Plan con IXSCAN sobre clabe_fecha y diez documentos examinados.

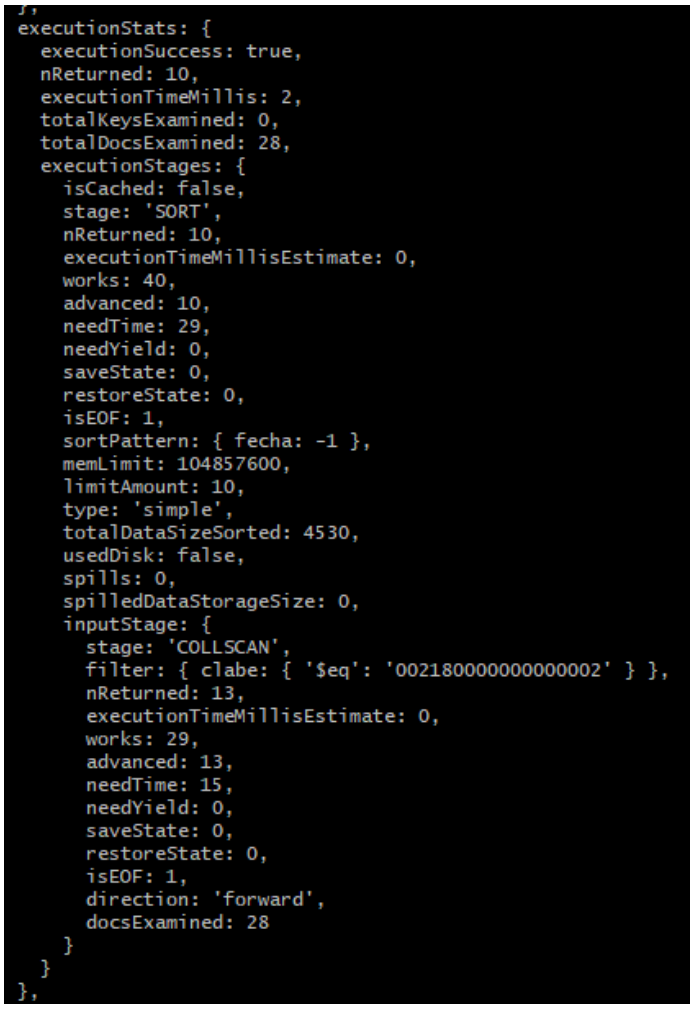



Figura 28: Plan forzado sin índice con COLLSCAN, SORT y 28 documentos examinados.

| Plan | Etapas | Llaves | Docs | Dev. | Tiempo ms |
|---|---|---|---|---|---|
| Con índice | IXSCAN FETCH LIMIT | 10 | 10 | 10 | 12 |
| Natural | COLLSCAN SORT | 0 | 28 | 10 | 2 |

En esta ejecución el tiempo registrado fue 12 ms con índice y 2 ms sin él. Eso no permite decir que el índice siempre sea más rápido ni que esté fallando: se trata de una colección pequeña y de una sola ejecución de cada plan. Lo que sí muestra la comparación es el trabajo realizado. El índice examinó diez documentos y resolvió el orden; el recorrido natural revisó 28 y necesitó ordenar. Esa diferencia se vuelve importante al crecer la colección.

# 8 Evento duplicado e idempotencia

RabbitMQ puede entregar un evento más de una vez. Para que eso no duplique los avisos, el _id de MongoDB combina eventoId y clabe. Usar sólo eventoId impediría guardar el segundo destinatario de una transferencia normal. Con la combinación, cada destinatario tiene su aviso único y una repetición genera exactamente la misma llave.

```
public int registrar(EventoTransferencia evento) {
        int nuevos = 0;

        for (Destinatario destinatario : destinatarios(evento)) {
                Document aviso = documento(
                                evento,
                                destinatario,
                                contactos.de(destinatario.clabe())
                );

                try {
                        mongo.insert(aviso, COLECCION);
                        nuevos++;
                } catch (DuplicateKeyException yaEstaba) {
                        log.info(
                                        "[avisos] {} ya estaba: evento repetido, no se duplica",
                                        aviso.get("_id")
                        );
                }
        }

        return nuevos;
}
```

registrar inserta uno por uno y captura DuplicateKeyException. Si el aviso ya existe, lo registra en el log y continúa con el siguiente destinatario. Esto también permite recuperar un evento si el proceso sólo alcanzó a guardar uno de sus dos avisos: al reintentarlo se omite el que ya estaba y se guarda el faltante. No se necesita que ambos inserts formen una sola transacción para obtener esa recuperación por destinatario.

Para comprobarlo se publicó dos veces el evento fijo 11111111-2222-4333-8444-555555555555, con CLABEs de terminación 091 y 092. Las dos publicaciones devolvieron routed: true, pero el conteo final en avisos fue de dos documentos, no cuatro. Además, los logs mostraron ya estaba para los avisos repetidos. Se aceptaron dos entregas del evento, pero no se repitió su efecto en la bitácora.

```
arroy@onofre MINGW64 ~/Documents/9no semestre/SD/sd-2027-1/entregas/arroyo_leonardo/s11/mexi-banco-s11 (trabajo-s11)
$ for i in 1 2; do
  echo "Publicacion $i"

  curl -s -u mexibanco:mexibanco \
    -X POST http://localhost:15672/api/exchanges/%2F/transferencias/publish \
    -H 'Content-Type: application/json' \
    -d '{"properties":{"delivery_mode":2,"content_type":"application/json"},"routing_key":"transferencia.realizada","p
\"clabeOrigen\":\"002180000000000091\",\"clabeDestino\":\"002180000000000092\",\"monto\":25,\"saldoOrigen\":475,\"sald

  echo
done
Publicacion 1
{"routed":true}
Publicacion 2
{"routed":true}

arroy@onofre MINGW64 ~/Documents/9no semestre/SD/sd-2027-1/entregas/arroyo_leonardo/s11/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval \
  'db.avisos.countDocuments({eventoId: "11111111-2222-4333-8444-555555555555"})'
2

arroy@onofre MINGW64 ~/Documents/9no semestre/SD/sd-2027-1/entregas/arroyo_leonardo/s11/mexi-banco-s11 (trabajo-s11)
$ docker compose logs notificacion | grep "ya estaba"
notificacion-1  | 2026-11-05T15:46:57.901Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos
notificacion-1  | 2026-11-05T15:46:57.903Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos

arroy@onofre MINGW64 ~/Documents/9no semestre/SD/sd-2027-1/entregas/arroyo_leonardo/s11/mexi-banco-s11 (trabajo-s11)
$
```

Figura 29: Dos publicaciones exitosas, conteo final de dos avisos y logs de deduplicación.

# 9 Saldo modificado directamente en Postgres

Para revisar el riesgo de una copia vieja se guardó el saldo de Ana y se consultó para llenar Redis. La respuesta inicial mostró 668. Después se actualizó directamente la fila en Postgres a 5, sin pasar por CuentaService y por lo tanto sin ejecutar su invalidación de caché.

En la secuencia capturada, la primera consulta posterior al UPDATE ya devolvió 5 y Redis mostró TTL igual a -2 antes de esa lectura: la llave no existía en ese momento. Las siguientes consultas continuaron mostrando 5, con cambios en leidaEn. El TTL configurado es de 30 segundos, pero esos 30 segundos son el tiempo de vida de una entrada, no una duración de dato viejo medida en esta secuencia. La salida permite distinguir la copia inicial de 668 de las lecturas posteriores que ya recuperaron el 5 de Postgres.

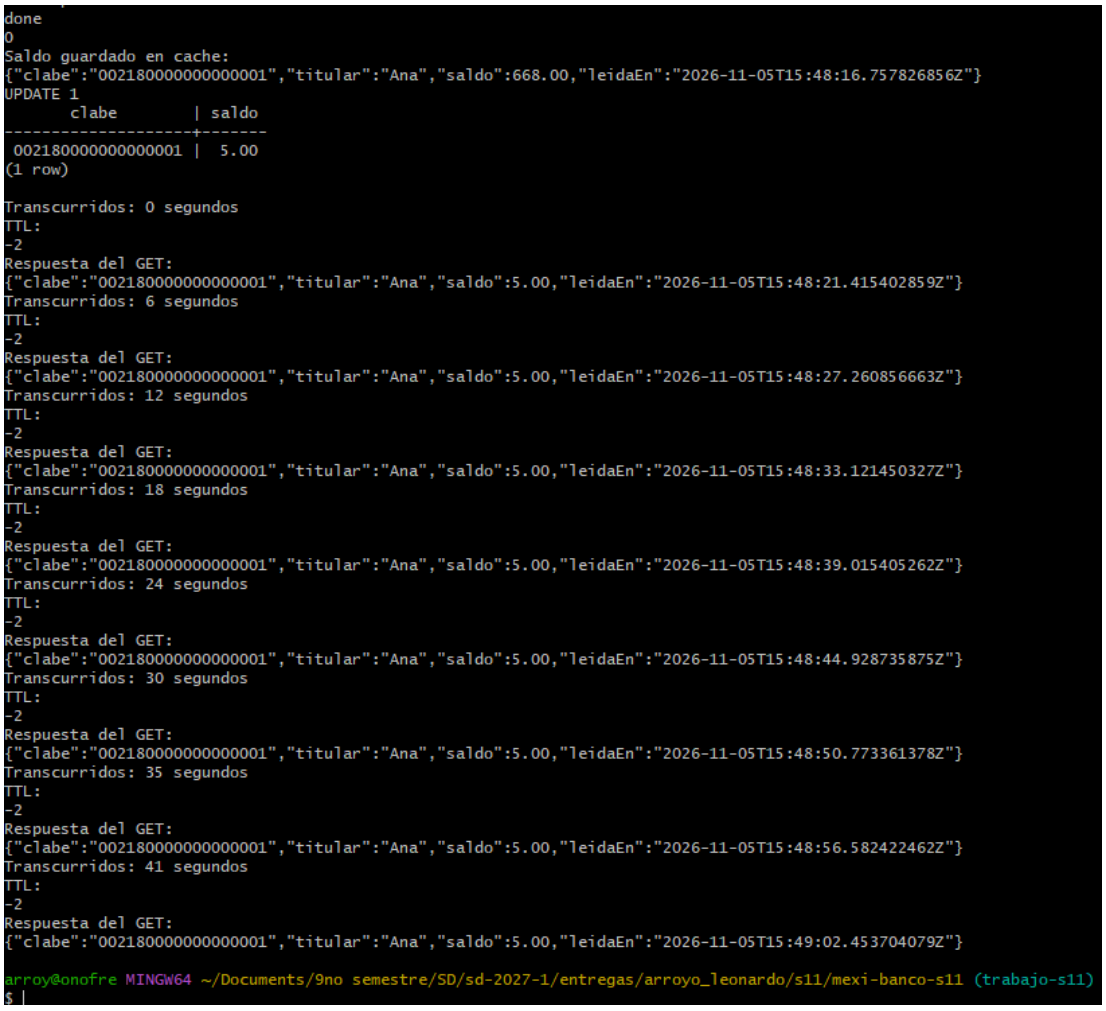



Figura 30: Saldo inicial consultado de 668, UPDATE a 5 y secuencia posterior de TTL y lecturas.

Al intentar transferir 100 la respuesta fue 422 Unprocessable Entity, con el detalle Saldo insuficiente en la CLABE de Ana. El cargo se autoriza con el repositorio de Postgres, donde sólo había 5. El dato que se muestra por GET puede provenir de Redis, pero no es la fuente que decide si se puede descontar dinero.

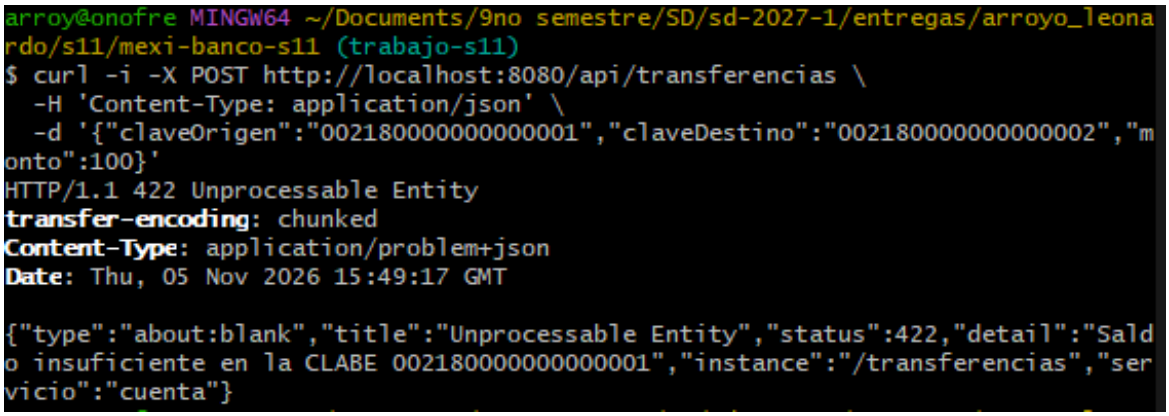



Figura 31: Transferencia de 100 rechazada con HTTP 422 por saldo insuficiente.

Al final se restauró el saldo guardado de 668 en Postgres, se borró la llave de Redis y se volvió a consultar. La respuesta mostró nuevamente 668 con otro leidaEn. Esto cerró la prueba controlada; el UPDATE directo se utilizó para provocar la diferencia y no es el flujo normal para modificar una cuenta.

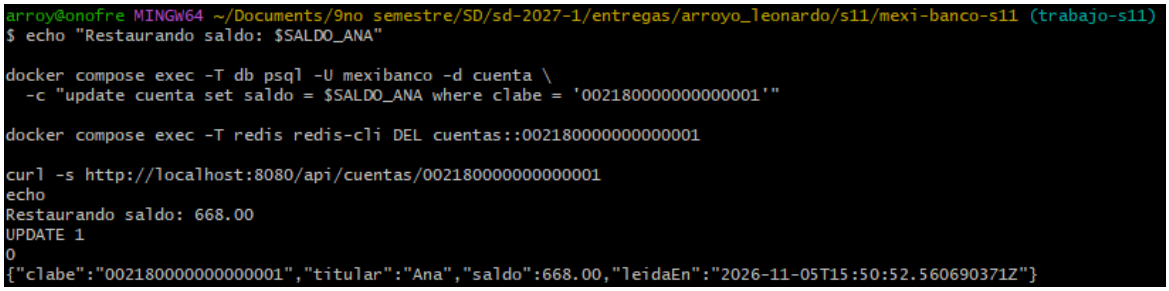



Figura 32: Restauración del saldo a 668 e invalidación de la copia de Redis.

# 10 Preguntas de cierre

¿Por qué MongoDB para los avisos y no para el saldo? Los avisos tienen campos distintos según SMS, CORREO o PUSH y se leen como documentos completos por cuenta y fecha. Ese acceso encaja con la colección y su índice. El saldo, en cambio, autoriza cargos y necesita conservar sus reglas dentro de la transacción de cuenta, con control de concurrencia e idempotencia. No se trata de que MongoDB sea incapaz de manejar transacciones, sino de que en este sistema el dinero ya tiene su autoridad en Postgres y no necesita cambiar de motor para guardar avisos.

¿Qué hit ratio se midió y qué se compró con la caché? En A se obtuvo 99 % y en B 83.33 %. Las transferencias invalidaron copias y aumentaron los misses. Con caché se midieron promedios de 10.37, 9.71 y 10.04 ms; sin caché, 18.87, 12.98 y 11.57 ms. Hubo una mejora de velocidad, pero el ahorro más claro fue pasar de 300 lecturas a Postgres a dos en los intervalos de trescientas consultas. Eso reduce trabajo repetido y favorece la capacidad de lectura, sin que la prueba mida un máximo de usuarios concurrentes.

¿Qué pasaría si el cargo autorizara con el saldo de Redis? Podría decidir con dinero que ya no existe. En el escenario del UPDATE, una copia de 668 no representaba los 5 guardados en Postgres. Si la autorización confiara sólo en esa copia, podría aprobar

una operación de 100 con una premisa incorrecta. Por eso la caché se usa para mostrar y CuentaService consulta la fuente real para aplicar las reglas del cargo.

¿Por qué DEL en lugar de SET y por qué después del commit? DEL descarta una copia que ya no debe tomarse como actual y deja que la siguiente lectura la reconstruya desde la base. Un SET calculado por una operación puede llegar después de otro y sobrescribir la caché con un saldo anterior. Además, si se invalida antes del commit, otro lector puede encontrar la llave vacía y volver a llenarla con el saldo que Postgres todavía no ha confirmado. Diferir la invalidación hasta después del commit reduce ese riesgo y evita publicar una copia de un cambio que termine en rollback. Aun así, la caché no se vuelve una fuente fuertemente consistente: sigue siendo una copia con TTL.

# Resultados y conclusión

Con esto se comprobó para qué sirve separar los almacenes. Postgres continuó decidiendo sobre el dinero; Redis permitió reutilizar una lectura entre tres instancias y reducir la carga repetida; MongoDB guardó los avisos con la forma de cada canal y permitió consultar los diez más recientes mediante un índice compuesto. La entrega repetida de un evento dejó dos avisos por sus dos destinatarios, sin duplicarlos.

Lo importante no fue solamente agregar dos bases. También hubo que definir qué dato puede estar atrasado, cuándo invalidarlo y cómo comprobar que un reintento no repita un efecto. Las mediciones y los planes muestran el beneficio concreto de cada decisión, mientras la prueba del saldo deja separadas la información que se muestra y la que se usa para autorizar una transferencia.